A MINI PROJECT REPORT ON

MARIO ADVENTURE – THE THREE WORLDS

Object-Oriented Programming (OOPs) Mini Project

Subject: Object-Oriented Programming

Course: Bachelor of Computer Applications (Data Science)

SUBMITTED BY		***SUBMITTED TO***	
Student Name:	Mahi Malhotra	***Teacher's Name:***	Mr. Abhinay Sood
UID:	25BCD10181	***Department:***	UIC
Course:	BCA(Data Science)	***College/University:***	Chandigarh University
Semester:	3rd sem		

Academic Session: 2026-27

1. Abstract

Mario Adventure – The Three Worlds is a two-dimensional graphical platform-adventure game developed in C++ as a mini project for the subject C++ / Object-Oriented Programming in the Bachelor of Computer Applications (BCA) course. The game is inspired by classic Mario-style platform games. It runs in its own Windows window and is not a console or ASCII game: the player, platforms, coins, enemies, backgrounds and on-screen text are all drawn with the Windows API and the Windows GDI (Graphics Device Interface).

The player controls a small character through three worlds named Forest World, Desert World and Ice Kingdom. In each world the player moves left and right, jumps onto platforms, collects coins to increase the score, and either avoids or jumps on moving enemies. The player starts with three lives, can reach a checkpoint in each world, and can use three temporary power-ups called Speed, Shield and Super Jump. The third world ends with a final boss who has three health points and must be defeated before the game can be completed. The game also contains a main menu, an instructions screen, a level-completion screen, a game-over screen, a victory screen and a restart option.

The complete program is written in a single source file, main.cpp. It uses structures, an enumeration, vectors, functions, global variables, conditions and loops, together with a timer-driven game loop, keyboard input and an off-screen drawing buffer for smooth graphics. The game is compiled with MinGW g++ and was developed in Visual Studio Code for Windows 10 / Windows 11. The program follows a structured, function-based style and does not define classes, so object-oriented features such as inheritance and polymorphism are not part of this version. The outcome of the project is a complete and playable game that helped the author practise core C++ programming, game logic, collision detection and Windows graphics programming.

Table of Contents / Index

1. Abstract.....2

2. Introduction.....4

3. Objectives of the Project.....4

4. Software and Hardware Requirements.....5

5. Technologies Used.....5

6. Working and Implementation of the Game.....7

7. Unique Features Added to the Game (UNIQUE) 11

8. Game Controls.....12

9. Complete Source Code.....13

10. Instructions to Compile and Run the Program.....26

11. Output Screenshots.....27

12. Conclusion.....34

13. Bibliography.....35

14. UML / Case Study.....36

2. Introduction

A 2D platform game is a type of video game in which the player controls a character who runs, jumps and moves across platforms while collecting items and avoiding dangers. Games of this kind became very popular on arcade machines and home consoles, and the Mario series is one of the best-known examples of the genre. The rules of a platform game are easy to understand, but they involve many useful programming ideas such as movement, gravity, collision detection, scoring, game states and graphics. This makes a small platform game a suitable mini project for a student who is learning C++. The purpose of this project is to apply the C++ topics studied in the course by building one complete game instead of several small, unrelated programs. ***Mario Adventure – The Three Worlds*** takes the familiar idea of a Mario-style platform game and builds a simple version of it with three different worlds, a score and lives system, moving enemies, a checkpoint, three temporary power-ups and a final boss. The original Mario game and its concept belong to their creators; this project is a separate version written for learning and practice.

The game is written in C++ and uses the Windows API to create and manage its window. A window procedure receives messages such as key presses, timer ticks and paint requests. A timer fires about every 16 milliseconds, and on each tick the program updates the game and redraws the screen. All pictures in the game are created with Windows GDI functions that draw rectangles, circles and text, so no image files are needed. An off-screen bitmap is used as a back buffer, so each frame is prepared first and then copied to the window in one step.

The program is organised using structures, functions and global variables, which are the C++ features that are actually used in the source code. Classes and objects are not used in the present version. This is explained honestly in the section on working and implementation so that the report matches the supplied source code exactly.

3. Objectives of the Project

The main objectives of this project are:

- To practise basic C++ features such as variables, constants, conditions, loops, functions, structures and enumerations.
- To use the standard library container vector to store platforms, coins and enemies.
- To understand the idea of a game loop, in which the game is updated and redrawn again and again.
- To implement player movement, gravity and jumping using keyboard input.
- To apply rectangle-based collision detection between the player, platforms, coins, enemies and the boss.
- To manage the score, the number of lives, the checkpoint, power-up timers and the boss health.
- To manage different screens (menu, instructions, playing, level complete, game over and win) using an enumeration named GameMode.
- To create graphical output using the Windows API and GDI instead of text-based output.
- To prepare a complete, working and well-documented mini project.

4. Software and Hardware Requirements

Software Requirements

Software	Purpose
Windows 10 or Windows 11	Operating system on which the game is developed and run. The program is Windows-specific.
Visual Studio Code	Code editor and integrated terminal used to write and compile the program.
MinGW g++ compiler	Compiles the C++ source file into a Windows executable (.exe) file.
C++ compiler support	The code uses auto, range-based for loops and brace initialisation, so a compiler with C++11 or later support is needed.
Windows API and GDI libraries	Header windows.h and libraries user32 and gdi32 provide the window, input and drawing functions.

Hardware Requirements

Hardware	Description
Windows-compatible computer	A desktop or laptop computer that can run Windows 10 or Windows 11.
Keyboard	Required to control the character, because the game uses keyboard input only.
Memory (RAM)	Minimum 4 GB RAM is recommended.
Graphics support	Basic graphics support is sufficient, because the game draws simple 2D shapes.
Storage	Sufficient storage for the source code and the compiled executable file.

5. Technologies Used

C++. C++ is the programming language used for the whole project. Besides the core language, the program uses a few parts of the standard library: vector for lists of game objects, string and stringstream for building text such as the score line, and cstring for the strlen function. The header algorithm is also included, although no function from it is called in the present version. ***Windows API.*** The Windows API (also called Win32) is the set of functions provided by Windows for creating windows and handling events. In this project it is used through windows.h. The program uses WinMain as the starting point, RegisterClassA and CreateWindowExA to create the window, a message loop with GetMessage, TranslateMessage and DispatchMessage, a window procedure called WindowProcedure, SetTimer for the game timer and GetAsyncKeyState to read the keyboard.

Windows GDI. GDI (Graphics Device Interface) is the part of Windows used for 2D drawing. The game uses CreateSolidBrush with FillRect to draw rectangles, Ellipse to draw circles, `CreateFontA with DrawTextA and TextOutA to write text, and CreateCompatibleDC`, CreateCompatibleBitmap and BitBlt to create and show the off-screen back buffer.

MinGW g++. MinGW provides the GNU C++ compiler (g++) for Windows. It compiles main.cpp and links it with the Windows libraries to create MarioAdventure.exe.

Visual Studio Code. Visual Studio Code is a free source-code editor. It was used to write the program and its integrated terminal was used to compile and run the game.

6. Working and Implementation of the Game

Window creation and start-up. The program starts in WinMain. It registers a window class named MarioAdventureWindow, creates a window titled "Mario Adventure - The Three Worlds" of size 900 by 600 pixels (with the maximise button removed), shows it, and then enters the standard message loop. All events are sent to the function WindowProcedure. When the window is created (WM_CREATE), the program creates an off-screen drawing buffer and starts a timer of 16 milliseconds. ***Game loop and game states.*** Every timer message (WM_TIMER) calls UpdateGame, which changes the game data, and then asks Windows to repaint the window. During painting (WM_PAINT) the function Paint draws the current screen into the back buffer and BitBlt copies it to the window. The current screen is stored in the variable mode, which is an enumeration of type GameMode with six values:
`MENU, INSTRUCTIONS, PLAYING, LEVEL_COMPLETE, GAME_OVER and WIN. Both UpdateGame and`
Paint check this variable, so only the code needed for the current screen is executed.

Main menu and instructions. The game opens on the main menu, which shows the title and the messages "Press ENTER to Start" and "Press I for Instructions". Pressing I opens the instructions screen (HOW TO PLAY), which lists the movement and jump keys and the basic aim of the game. Enter starts the game from either screen, and Esc returns to the main menu.

Keyboard input. Two methods are used. The movement and jump keys (Left, Right, Space and Up) are checked continuously with GetAsyncKeyState inside UpdateGame, so the character keeps moving while a key is held down. One-time commands such as Enter, I, R, Esc, S, H and J are handled
`in the WM_KEYDOWN case of WindowProcedure.`
Player movement, gravity and jumping. The player's position is stored in playerX and playerY, and the vertical speed is stored in velocityY. The normal walking speed is 5 pixels per update. Pressing a jump key while the player is standing on a surface (onGround is true) gives an upward speed of -16. In every update gravity adds 1 to velocityY, and the falling speed is limited to 15. The world is 2300 pixels wide, and the player cannot walk outside it. The value cameraX follows the player so that the 900-pixel-wide window always shows the part of the world around the character. ***Platforms and collision detection.*** Platforms are stored in a vector of the structure Platform. The ground is a long platform at the bottom of the world, and each world adds five floating platforms. The function Collision checks whether two rectangles overlap by comparing their edges; this is the basic form of rectangle (bounding box) collision detection. When the falling player overlaps a platform, the player is placed on top of it and onGround becomes true. If the player is moving upwards and hits the underside of a platform, the player is pushed below it and the upward movement stops.

Coin collection and score. Every world contains 20 coins, which are placed by a loop in LoadStage in a repeating pattern of three heights. When the player's rectangle overlaps a coin, the coin is marked as collected and 10 points are added to the score. The other score rewards are 20 points for defeating an enemy, 50 points for reaching the checkpoint, 50 points for each successful hit on the boss and 200 extra points for defeating the boss. The score, lives and world number are shown at the top-left corner of the screen.

Enemy movement. Enemies are stored in a vector of the structure Enemy. Each enemy has a left limit and a right limit and moves 2 pixels per update between them, turning back at each limit. World 1 and

World 2 have three enemies each, and World 3 has two. If the player lands on an enemy from above, the enemy is defeated, the player bounces upward and 20 points are added. If the player touches an enemy in any other way, the function LoseLife is called.

Lives and checkpoints. The player starts the game with three lives. LoseLife reduces the number of lives by one. If lives remain, the player is placed back at the starting position (x = 80) or, if the checkpoint has been reached, at the checkpoint position (x = 1100). When the player's x position passes 1050 for the first time in a world, the checkpoint is saved, 50 points are added and the message CHECKPOINT SAVED is displayed. The code also contains a test for falling below the screen, but in the present levels the ground is continuous, so lives are mainly lost by touching enemies or the boss. When the number of lives reaches zero, the game changes to the game-over screen.

Power-ups. The game has three temporary power-ups. In the supplied code they are activated by pressing S (Speed), H (Shield) or J (Super Jump) during play. The active power-up is stored in powerType and lasts for powerTimer = 300 updates, which is about five seconds. Speed raises the walking speed from 5 to 9 pixels per update. Shield makes LoseLife return without removing a life and draws a light-blue circle around the player. Super Jump raises the jump speed from -16 to -22. Only one power-up is active at a time, and its name is shown on the screen. In this version the power-ups are triggered by key presses; there are no power-up items placed in the levels.

Boss health and the final battle. In World 3 a purple boss is placed near the end of the level. It moves slowly between x = 1800 and x = 1980 and has a health value bossHealth of 3, which is shown above it as BOSS HP. Jumping on the boss from above reduces its health by one, makes the player bounce, adds 50 points and starts a short cooldown of 35 updates so that one landing is not counted several times. When the health reaches zero the boss disappears and 200 bonus points are added. Touching the boss in any other way calls LoseLife. While the boss is alive, the message DEFEAT THE BOSS FIRST is displayed and World 3 cannot be completed.

Level transitions, game over and victory. Each world ends with a finish flag at x = 2140, drawn as a white pole with a red flag. Reaching it in World 1 or World 2 calls CompleteStage; in World 3 it works only after the boss has been defeated. The game then shows a level-complete screen (for example, WORLD 1 COMPLETED!) for 100 updates, about 1.6 seconds, before NextStage loads the next world. After World 3 the victory screen appears with the final score. If the lives reach zero, the game-over screen appears instead. On both screens, R restarts the game and Esc returns to the main menu.

Graphics and appearance of the worlds. Everything on the screen is drawn with GDI. DrawRectangle fills a rectangle with a solid brush, DrawCircle draws an ellipse, DrawTextCenter creates an Arial font and writes centred text, and TextOutA writes the score line. The player is made from rectangles and a small circle, enemies are red circles with white eyes, coins are yellow circles and the finish flag is made from two rectangles. The Forest World has a light-blue sky and green trees, the Desert World has an orange sky and a yellow sun, and the Ice Kingdom has a pale-blue sky with pale circles that look like snow mounds. The platforms use the same brown body and green top in every world. Emoji characters are not used, because they did not display properly in the Windows graphics environment; shapes drawn with GDI are used instead.

C++ concepts used in the program

Because this is a C++ / OOPssubject project, it is important to be clear about which concepts are actually present in the source code. The program is written in a structured (procedural) style with some C++ features, and it does not define any classes. The concepts that are implemented are listed below.

Concept	*Where it is used in the program*
Structures (`struct`)	Platform, Coin and Enemy group related data such as position, size, limits and status.
Enumeration (enum)	GameMode stores which screen of the game is active.
Standard library vector	`platforms, coins and enemies are vectors; push_back and` are used to build each world.
Functions	The program is divided into 22 functions with parameters and return values; for example Collision returns a bool value.
Global variables and constants	Game data such as score, lives and playerX are global variables;
Conditions	`SCREEN_W and WORLD_W are const int constants. switch` statement handles window messages. if, else if and else statements control the game rules, and a
Loops	for loops create coins and background objects, range-based for loops go through the vectors, and a while loop runs the message loop.
References and`uto`	for (auto& c : coins) changes the original coin, while for `(auto p : platforms) works on a copy.`
Strings and streams	stringstream, string and const char* are used to prepare text shown on the screen.
Callback function	WindowProcedure is called by Windows for every window message.

The following concepts belong to the broader theory of object-oriented programming, but they are ***implemented*** in this version of the program.

OOP concept	*Status in this program*
Class and object	Not used. The structures contain only data members; they have no member functions, constructors or access control.
Encapsulation	Not implemented. Game data is stored in global variables and public structure members.
Inheritance	Not used. No class is derived from another class.
Polymorphism	Not used. The program has no virtual functions.
Abstraction	Not implemented through classes or interfaces. Functions only divide the work into smaller tasks.

The three structures are still a useful first step towards object-oriented design, because they already group related data together. In a future version, they could be converted into classes with their own member functions. This is a possible improvement and is not part of the present program.

Functions in the program

Function	Purpose
Collision	Returns true if two rectangles overlap.
AddPlatform	Adds one platform to the platforms vector.
AddCoin	Adds one coin to the coins vector.
AddEnemy	Adds one enemy with its left and right limits to the enemies vector.
LoadStage	Clears old objects and creates the platforms, enemies, coins and boss data for the current world.
StartGame	Resets score, lives and world number, loads World 1 and starts play.
NextStage	Moves to the next world, or to the victory screen after World 3.
CompleteStage	Starts the level-complete screen and its timer.
LoseLife	Removes one life (unless the shield is active), respawns the player or ends the game.
DrawRectangle	Draws a filled rectangle using a solid brush.
DrawCircle	Draws a filled circle using Ellipse.
DrawTextCenter	Writes centred text with a chosen size and colour.
DrawPlayer	Draws the player and the shield circle when the shield is active.
DrawGame	Draws the background, platforms, coins, enemies, boss, finish flag and the score line.
UpdateGame	Handles input, movement, gravity, collisions, enemies, checkpoint, boss, power-up timer, camera and finish check.
DrawMenu	Draws the main menu screen.
DrawInstructions	Draws the instructions screen.
Paint	Chooses which screen to draw according to the current mode.
CreateBuffer	Creates the off-screen bitmap used for double buffering.
DeleteBuffer	Releases the off-screen bitmap and device context.
WindowProcedure	Receives window messages (create, timer, key press, paint, destroy). Program entry point: creates the window and runs the message loop.
WinMain	

7. UNIQUE FEATURES ADDED TO THE GAME

The idea of a Mario-style platform game is not new, and this project does not claim to have created it.

The features described below are the ***additional features implemented in this version of the game*** to make it more varied and interesting than a very basic platform game. Each feature listed here is present in the supplied source code.

Feature	*Description and benefit to the player*
Three separate worlds	The game has Forest World, Desert World and Ice Kingdom. LoadStage builds a different layout of platforms and enemies for each world, so the player faces new challenges instead of repeating one level.
Different graphical backgrounds	Each world has its own sky colour and decoration: green trees in the Forest, a sun in the Desert and pale snow mounds in the Ice Kingdom. This gives every world a clearly different look.
Three-life system	The player starts with three lives. This allows mistakes while still making every enemy contact important.
Moving enemies with collision detection	Enemies patrol between fixed limits. Landing on an enemy from above defeats it and gives 20 points, while touching it from the side loses a life. The player must judge timing and position.
Checkpoint system	Passing x = 1050 saves a checkpoint, gives 50 points and shows CHECKPOINT SAVED. After losing a life the player restarts from the checkpoint instead of the beginning.
Three temporary power-ups	Speed, Shield and Super Jump each last about five seconds. They add variety to movement and help the player to cross difficult parts or avoid damage.
Final boss battle with health	World 3 has a boss with three health points. It must be stomped three times before the exit works, which makes the last world feel different from the first two.
Transition screens between worlds	A separate screen shows WORLD n COMPLETED! before the next world starts, so the player can see the progress through the game
Menu, instructions, game-over and victory screens	Separate screens explain the controls, show the result of the game and allow the player to restart with R or return to the menu with Esc.
Score rewards	Coins give 10 points, enemies 20, the checkpoint 50, each boss hit 50 and the defeated boss 200 more. This gives the player a reason to collect and fight.
Custom icons drawn with GDI	The character, enemies, coins, trees, sun, snow mounds and finish flag are drawn using rectangles and circles instead of emoji characters, which did not display properly in the Windows graphics environment.

Together, these features make the game more engaging than a basic platform game with one level. The different worlds and the boss give the player goals to work towards, the score and lives create a sense of risk and reward, and the checkpoint and power-ups reduce frustration while still keeping the game challenging.

8. Game Controls

Key	Function
Enter	Start the game (from the main menu or the instructions screen)
I	Open the instructions screen (from the main menu)
Left Arrow	Move left
Right Arrow	Move right
Space / Up Arrow	Jump
R	Restart the game from World 1 with score 0 and three lives
Esc	Return to the main menu
S	Activate the Speed power-up during play
H	Activate the Shield power-up during play
J	Activate the Super Jump power-up during play

Power-ups. The game has three power-ups, and each one lasts for about five seconds after it is activated. In the supplied code the power-ups are activated by the keys S, H and J during gameplay; the levels do not contain collectible power-up objects, and these three keys are not listed on the in-game instructions screen.

- ***Speed (S):*** the walking speed increases from 5 to 9 pixels per update.
- ***Shield (H):*** contact with enemies or the boss does not remove a life, and a light-blue circle is drawn around the player.

•***Super Jump (J):*** the jump speed increases from -16 to -22, so the player jumps higher.

Only one power-up can be active at a time. The name of the active power-up is displayed under the score line while it lasts.

9. Complete Source Code

The complete source code of the game is given below. It is a single file `main.cpp` . The listing named is the full program and has not been shortened or changed.

```
#include <windows.h>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
#include <cstring>
using namespace std;

const int SCREEN_W = 900;
const int SCREEN_H = 600;
const int WORLD_W = 2300;

const int PLAYER_W = 38;
const int PLAYER_H = 48;

const int EXIT_X = 2140;
const int BOSS_Y = 485;

HWND hwnd;
HDC backDC;
HBITMAP backBitmap;
HBITMAP oldBitmap;

enum GameMode {
    MENU,
    INSTRUCTIONS,
    PLAYING,
    LEVEL_COMPLETE,
    GAME_OVER,
    WIN
};

GameMode mode = MENU;

int playerX = 80;
int playerY = 450;
int velocityY = 0;
int previousBottom = 0;

int cameraX = 0;
int  score  =  0;
int  lives  =  3;
int stage = 0;

int powerTimer = 0;
int powerType = 0;

int transitionTimer = 0;

int bossX = 1850;
int bossHealth = 3;
int bossDirection = 1;
int bossCooldown = 0;

bool onGround = false;
bool checkpointReached = false;
bool bossAlive = true;

struct Platform {
```

```
    int x, y, w, h;
};

struct Coin {
 int x, y;
 bool collected;
};

struct Enemy {
    int x, y;
    int left, right;
    int direction;
    bool alive;
};

vector<Platform> platforms;
vector<Coin> coins;
vector<Enemy> enemies;

bool Collision(int x1, int y1, int w1, int h1,
               int x2, int y2, int w2, int h2) {
    return x1 < x2 + w2 &&
    x1 + w1 > x2 && y1 <
    y2 + h2 && y1 + h1 >
    y2;
}

void AddPlatform(int x, int y, int w, int h) {
    platforms.push_back({x, y, w, h});
}

void AddCoin(int x, int y) {
    coins.push_back({x, y, false});
}

void AddEnemy(int x, int left, int right) {
    enemies.push_back({x, 510, left, right, 1, true});
}

void LoadStage() {
    platforms.clear();
    coins.clear();
    enemies.clear();

    playerX  =  80;
    playerY = 450;
    velocityY = 0;
    cameraX = 0;

    onGround = false;
    checkpointReached = false;

    powerTimer = 0;
    powerType = 0;

    bossAlive = (stage == 2);
    bossHealth = 3;
    bossX = 1850;
    bossDirection = 1;
    bossCooldown = 0;


    AddPlatform(0, 550, WORLD_W, 50);

    if (stage == 0) {
        AddPlatform(300, 450, 150, 20);
        AddPlatform(600, 390, 150, 20);
        AddPlatform(950, 450, 160, 20);
```

```
        AddPlatform(1300, 390, 150, 20);
        AddPlatform(1650, 450, 180, 20);

        AddEnemy(500,   450,   750);
        AddEnemy(1150, 1000, 1400);
        AddEnemy(1750, 1650, 1900);
    }
    elseif (stage == 1) {
        AddPlatform(250,  430,  150,  20);
        AddPlatform(550,  360,  170,  20);
        AddPlatform(900,  430,  180,  20);
        AddPlatform(1250, 350,  160,  20);
        AddPlatform(1650, 420, 180, 20);

        AddEnemy(450, 350, 650);
        AddEnemy(1000, 900, 1200);
        AddEnemy(1750, 1600, 1900);
    }
    else{
        AddPlatform(280,  420,  150,  20);
        AddPlatform(600,  350,  150,  20);
        AddPlatform(950,  420,  180,  20);
        AddPlatform(1300, 350,  160, 20);
        AddPlatform(1700, 420, 180, 20);

        AddEnemy(500, 400, 700);
        AddEnemy(1150, 1000, 1350);
    }

    for(int i = 0; i < 20; i++) {
        AddCoin(180 + i * 95, 480 - (i % 3) * 65);
    }
}

void StartGame() {
    score = 0;
    lives = 3;
    stage      =      0;
    transitionTimer = 0;

    LoadStage();
    mode = PLAYING;
}

void NextStage() {
    stage++;

    if (stage >= 3) {
        mode = WIN;
        return;
    }

    LoadStage();
    mode = PLAYING;
}

void CompleteStage() {
    if (mode != PLAYING) return;

    mode = LEVEL_COMPLETE;
    transitionTimer = 100;
}

voidLoseLife() {
    if (powerType == 2 && powerTimer > 0) {
        return;
    }
```

```
    lives--;

    if (lives <= 0) {
        mode = GAME_OVER;
        return;
    }

    playerX = checkpointReached ? 1100 : 80;
    playerY = 450;
    velocityY = 0;
    onGround = false;
}

void DrawRectangle(HDC dc, int x, int y, int w, int h,
                   COLORREF color) {
    HBRUSH brush = CreateSolidBrush(color);
    RECT r = {x, y, x + w, y + h};
    FillRect(dc, &r, brush);
    DeleteObject(brush);
}

void DrawCircle(HDC dc, int x, int y, int r, COLORREF color) {
    HBRUSH brush = CreateSolidBrush(color);
    HBRUSH oldBrush = (HBRUSH)SelectObject(dc, brush);

    Ellipse(dc, x - r, y - r, x + r, y + r);

    SelectObject(dc, oldBrush);
    DeleteObject(brush);
}

void DrawTextCenter(HDC dc, const char* text, int y,
                    int size, COLORREF color) {
    SetTextColor(dc, color);
    SetBkMode(dc, TRANSPARENT);

    HFONTfont = CreateFontA(
        size, 0, 0, 0, FW_BOLD,
        FALSE,   FALSE,   FALSE,
        DEFAULT_CHARSET,
        OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS,
        ANTIALIASED_QUALITY,
        DEFAULT_PITCH,
        "Arial"
    );

    HFONT oldFont = (HFONT)SelectObject(dc, font);

    RECT r = {0, y, SCREEN_W, y + 60};
    DrawTextA(dc, text, -1, &r, DT_CENTER | DT_SINGLELINE);

    SelectObject(dc, oldFont);
    DeleteObject(font);
}

void DrawPlayer(HDC dc) {
    int x = playerX - cameraX;

    DrawRectangle(dc, x + 5, playerY, 28, 20,
                  RGB(220, 40, 40));

     DrawRectangle(dc, x + 3, playerY + 18, 32, 22,
                   RGB(30, 80, 220));

     DrawRectangle(dc, x + 7, playerY + 40, 10, 8,
                   RGB(80, 40, 20));
```

```
    DrawRectangle(dc, x + 23, playerY + 40, 10, 8,
                  RGB(80, 40, 20));

    DrawCircle(dc, x + 19, playerY + 9, 3,
               RGB(255, 255, 255));

    if    (powerType==2 &&powerTimer > 0) {
        DrawCircle(dc, x+  19, playerY + 24, 30,
                   RGB(100, 220, 255));
    }
}

void DrawGame(HDC dc) {
    COLORREF sky;
    COLORREF ground;

    if (stage == 0) {
        sky = RGB(120, 210, 255);
        ground = RGB(30, 150, 60);
    }
    elseif (stage == 1) {
        sky = RGB(255, 190, 100);
        ground = RGB(205, 140, 55);
    }
    else{
        sky = RGB(130, 190, 255);
        ground = RGB(180, 230, 255);
    }

    DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H, sky);

    if (stage==0){
        for(inti   =0;i<    SCREEN_W; i += 180) {
            DrawCircle(dc, i + 40, 440, 35,
                       RGB(30, 130, 40));

            DrawRectangle(dc, i + 32, 440, 16, 80,
                          RGB(110, 65, 30));
        }
    }
    else if (stage == 1) {
        DrawCircle(dc, 760, 100, 45, RGB(255, 220, 40));
    }
    else{
        for(inti   =0;i<    SCREEN_W; i += 150) {
             DrawCircle(dc, i + 50, 500, 35,
                        RGB(230, 250, 255));
        }
    }

    DrawRectangle(dc, 0, 550, SCREEN_W, 50, ground);

      for(auto p : platforms) {
         int x = p.x - cameraX;

        if (x+p.w>0&&      x < SCREEN_W) {
            DrawRectangle(dc, x, p.y, p.w, p.h,
                          RGB(100, 65, 35));

            DrawRectangle(dc, x, p.y, p.w, 6,
                          RGB(40, 180, 60));
        }
    }

    for(autoc:coins) {
        if (!c.collected) {
            intx=c.x - cameraX;
```

```
        if (x>-30&&    x < SCREEN_W + 30) {
            DrawCircle(dc, x, c.y, 10,
                       RGB(255, 215, 0));
        }
    }
}

for(autoe:enemies) {
    if (e.alive) {
        intx=e.x - cameraX;

        DrawCircle(dc, x + 15, e.y + 15, 17,
                   RGB(180, 40, 40));

        DrawCircle(dc, x + 9, e.y + 10, 3,
                   RGB(255, 255, 255));

        DrawCircle(dc, x + 21, e.y + 10, 3,
                   RGB(255, 255, 255));
    }
}

if (stage == 2 && bossAlive) {
    int x = bossX - cameraX;

    DrawRectangle(dc, x, BOSS_Y, 72, 82,
                  RGB(100, 20, 150));

    DrawCircle(dc, x + 20, 510, 7,
               RGB(255, 255, 255));

    DrawCircle(dc, x + 52, 510, 7,
               RGB(255, 255, 255));

    DrawRectangle(dc, x + 10, 470, 52, 12,
                  RGB(70, 10, 100));

    stringstream ss;
    ss << "BOSS HP: " << bossHealth;
    string s = ss.str();

    SetTextColor(dc, RGB(255, 255, 255));
    TextOutA(dc, x, 455, s.c_str(), (int)s.size());
}

int exitScreenX = EXIT_X - cameraX;

DrawRectangle(dc, exitScreenX, 490, 18, 60,
              RGB(255, 255, 255));

DrawRectangle(dc, exitScreenX + 18, 490, 45, 25,
              RGB(255, 40, 40));

DrawPlayer(dc);

stringstream ss;
ss << "Score: " << score
   <<  "   Lives: " << lives
   << "    World: " << stage + 1;

string hud = ss.str();

SetTextColor(dc, RGB(0, 0, 0));
SetBkMode(dc, TRANSPARENT);
TextOutA(dc, 15, 15, hud.c_str(), (int)hud.size());

if (powerTimer > 0) {
```

```
        const char* powerName = "POWER ACTIVE";

        if (powerType == 1) powerName = "SPEED POWER";
        if (powerType == 2) powerName = "SHIELD POWER";
        if (powerType == 3) powerName = "SUPER JUMP";

        TextOutA(dc, 15, 40, powerName,
                 (int)strlen(powerName));
    }

    if (checkpointReached) {
        TextOutA(dc, 700, 15, "CHECKPOINT SAVED", 16);
    }

    if (stage==2   &&bossAlive) {
        TextOutA(dc,300, 15,
                 "DEFEAT THE BOSS FIRST", 21);
    }
}

void UpdateGame() {
    if (mode == LEVEL_COMPLETE) {
        transitionTimer--;

        if (transitionTimer <= 0) {
            NextStage();
        }

        return;
    }

    if (mode != PLAYING) return;

    int speed = 5;

    if (powerType == 1 && powerTimer > 0) {
        speed = 9;
    }

    if (GetAsyncKeyState(VK_LEFT) & 0x8000) {
        playerX -= speed;
    }

    if (GetAsyncKeyState(VK_RIGHT) & 0x8000) {
        playerX += speed;
    }

    if (playerX < 0) playerX = 0;

    if (playerX > WORLD_W - PLAYER_W) {
        playerX = WORLD_W - PLAYER_W;
    }

    if ((GetAsyncKeyState(VK_SPACE) & 0x8000 ||
         GetAsyncKeyState(VK_UP) & 0x8000) && onGround) {

        velocityY = -16;

        if (powerType == 3 && powerTimer > 0) {
            velocityY = -22;
        }

        onGround = false;
    }

    previousBottom = playerY + PLAYER_H;
    velocityY += 1;
```

```
if (velocityY > 15) velocityY = 15;

playerY += velocityY;
onGround = false;

for(autop:platforms) {
    if (Collision(playerX, playerY, PLAYER_W, PLAYER_H,
                  p.x, p.y, p.w, p.h)) {

        if (velocityY > 0) {
            playerY = p.y - PLAYER_H;
            velocityY = 0;
            onGround = true;
        }
        elseif (velocityY < 0) {
            playerY = p.y + p.h;
            velocityY = 0;
        }
    }
}

if (playerY > SCREEN_H) {
    LoseLife();
    return;
}

for(auto&  c:  coins) {
    if (!c.collected &&
        Collision(playerX, playerY, PLAYER_W, PLAYER_H,
                  c.x - 10, c.y - 10, 20, 20)) {

        c.collected = true;
        score += 10;
    }
}

for(auto& e : enemies) {
    if (!e.alive) continue;

    e.x += e.direction * 2;

    if (e.x < e.left || e.x > e.right) {
        e.direction *= -1;
    }

    if (Collision(playerX, playerY, PLAYER_W, PLAYER_H,
                  e.x, e.y, 30, 30)) {

        if (velocityY > 0 &&
            previousBottom <= e.y + 20) {

            e.alive = false;
            velocityY = -10;
            score += 20;
        }
        else{
            LoseLife();
            return;
        }
    }
}

if (playerX > 1050 && !checkpointReached) {
    checkpointReached = true;
    score += 50;
}
if (stage == 2 && bossAlive) {
```

```
        if (bossCooldown > 0) {
            bossCooldown--;
        }

        bossX += bossDirection * 2;

        if (bossX < 1800 || bossX > 1980) {
            bossDirection *= -1;
        }

        booltouchingBoss =
            Collision(playerX, playerY, PLAYER_W, PLAYER_H,
                      bossX, BOSS_Y, 72, 82);

        boolstompingBoss =
            velocityY > 0 &&
            previousBottom <= BOSS_Y + 20;

        if (touchingBoss && bossCooldown == 0) {
            if (stompingBoss) {
                bossHealth--;
                velocityY = -14;
                score += 50;
                bossCooldown = 35;

                if (bossHealth <= 0) {
                   bossAlive  =  false;
                   score += 200;
                }
            }
            else{
                LoseLife();
                return;
            }
        }
    }

    if (powerTimer > 0) {
        powerTimer--;
    }
    else{
        powerType = 0;
    }

    cameraX = playerX - SCREEN_W / 2;

    if (cameraX < 0) cameraX = 0;

    if (cameraX > WORLD_W - SCREEN_W) {
        cameraX = WORLD_W - SCREEN_W;
    }

    // FINISH CHECK: reach the exit area.
    // World 1 and 2 finish immediately.
    // World 3 finishes only after defeating the boss.

    if  (playerX + PLAYER_W >= EXIT_X) {
        if  (stage != 2 || !bossAlive) {
            CompleteStage();
        }
    }
}

void DrawMenu(HDC dc) {
    DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H,
                  RGB(80, 160, 240));
    DrawTextCenter(dc, "MARIO ADVENTURE", 120, 48,
```

```
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "THE THREE WORLDS", 190, 25,
                    RGB(255, 240, 0));

    DrawTextCenter(dc, "Press ENTER to Start", 300, 25,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "Press I for Instructions", 350, 20,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "C++ Game Project", 480, 18,
                    RGB(255, 255, 255));
}

void DrawInstructions(HDC dc) {
    DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H,
                  RGB(40, 80, 130));

    DrawTextCenter(dc, "HOW TO PLAY", 80, 38,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "LEFT / RIGHT ARROW : Move", 160, 21,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "SPACE / UP ARROW : Jump", 205, 21,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "Collect coins to increase your score", 250, 19,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "Reach the finish flag to change worlds", 295, 19,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "Defeat the boss in World 3", 340, 19,
                    RGB(255, 255, 255));

    DrawTextCenter(dc, "Press ENTER to Start", 430, 23,
                    RGB(255, 240, 0));

     DrawTextCenter(dc, "Press ESC to Return", 480, 18,
                     RGB(255, 255, 255));
}

void Paint(HDC dc) {
    if (mode == MENU) {
        DrawMenu(dc);
    }
    else if (mode == INSTRUCTIONS) {
        DrawInstructions(dc);
    }
    else if (mode == PLAYING) {
        DrawGame(dc);
    }
    else if (mode == LEVEL_COMPLETE) {
        DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H,
                      RGB(20, 120, 70));

        stringstream ss;
        ss << "WORLD " << stage + 1 << " COMPLETED!";

        string message = ss.str();

        DrawTextCenter(dc, message.c_str(), 180, 42,
                       RGB(255, 255, 255));

        DrawTextCenter(dc, "Get Ready for the Next World!", 270, 25,
```

```
                        RGB(255, 240, 0));
    }
    elseif(mode==GAME_OVER) {
        DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H,
                      RGB(80, 20, 20));

        DrawTextCenter(dc, "GAME OVER", 180, 50,
                       RGB(255, 255, 255));

        DrawTextCenter(dc, "Press R to Restart", 300, 25,
                       RGB(255, 240, 0));

        DrawTextCenter(dc, "Press ESC for Menu", 350, 20,
                       RGB(255, 255, 255));
    }
    elseif(mode==WIN) {
        DrawRectangle(dc, 0, 0, SCREEN_W, SCREEN_H,
                      RGB(20, 130, 70));

        DrawTextCenter(dc, "YOU WIN!", 150, 55,
                       RGB(255, 255, 255));

        stringstream ss;
        ss << "Final Score: " << score;

        string result = ss.str();

        DrawTextCenter(dc, result.c_str(), 250, 28,
                       RGB(255, 240, 0));

        DrawTextCenter(dc, "You Completed All Three Worlds!", 320, 24,
                       RGB(255, 255, 255));

        DrawTextCenter(dc, "Press R to Play Again", 390, 23,
                       RGB(255, 255, 255));

        DrawTextCenter(dc, "Press ESC for Menu", 440, 20,
                       RGB(255, 255, 255));
    }
}

void CreateBuffer(HWND window) {
    HDC dc = GetDC(window);

    backDC = CreateCompatibleDC(dc);
    backBitmap = CreateCompatibleBitmap(dc, SCREEN_W, SCREEN_H);

    oldBitmap = (HBITMAP)SelectObject(backDC, backBitmap);

    ReleaseDC(window, dc);
}

voidDeleteBuffer() {
    if (backDC) {
        SelectObject(backDC, oldBitmap);
        DeleteObject(backBitmap);
        DeleteDC(backDC);
    }
}

LRESULT CALLBACK WindowProcedure(HWND window, UINT message,
                                 WPARAM wParam, LPARAM lParam) {
    switch (message) {
    case WM_CREATE:
        CreateBuffer(window);
        SetTimer(window, 1, 16, NULL);
        return 0;
```

```
    case WM_TIMER:
        UpdateGame();
        InvalidateRect(window, NULL, FALSE);
        return 0;

    case WM_KEYDOWN:
        if (wParam == VK_RETURN) {
            if (mode == MENU || mode == INSTRUCTIONS) {
                StartGame();
            }
        }

        if (wParam == 'I' && mode == MENU) {
            mode = INSTRUCTIONS;
        }

        if (wParam == 'R') {
            StartGame();
        }

        if (wParam == VK_ESCAPE) {
            mode = MENU;
        }

        if (mode== PLAYING) {
            if (wParam == 'S') {
                powerType = 1;
                powerTimer = 300;
            }

            if (wParam == 'H') {
                powerType = 2;
                powerTimer = 300;
            }

            if (wParam == 'J') {
                powerType = 3;
                powerTimer = 300;
            }
        }

        InvalidateRect(window, NULL, FALSE);
        return 0;

    caseWM_PAINT: {
        PAINTSTRUCT ps;
        HDC dc = BeginPaint(window, &ps);

        Paint(backDC);

        BitBlt(dc, 0, 0, SCREEN_W, SCREEN_H,
               backDC, 0, 0, SRCCOPY);

        EndPaint(window, &ps);
        return 0;
    }

    case WM_DESTROY:
        KillTimer(window, 1);
        DeleteBuffer();
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(window, message, wParam, lParam);
}
int WINAPI WinMain(HINSTANCE instance, HINSTANCE previous,
```

```
                LPSTR commandLine, int showCommand) {
    constchar  CLASS_NAME[] = "MarioAdventureWindow";

    WNDCLASSA wc = {};

    wc.lpfnWndProc = WindowProcedure;
    wc.hInstance = instance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);

    RegisterClassA(&wc);

    hwnd = CreateWindowExA(
        0,
        CLASS_NAME,
        "Mario Adventure - The Three Worlds",
        WS_OVERLAPPEDWINDOW & ~WS_MAXIMIZEBOX,
        CW_USEDEFAULT, CW_USEDEFAULT,
        SCREEN_W, SCREEN_H,
        NULL, NULL, instance, NULL
    );

    if (!hwnd) return 0;

    ShowWindow(hwnd, showCommand);
    UpdateWindow(hwnd);

    MSG msg = {};

    while(GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return 0;
}
```

10. Instructions to Compile and Run the Program

This project is designed for Windows 10 and Windows 11 because it uses the Windows API and GDI graphics. It cannot be compiled or run on other operating systems without changes.

Steps

1. ***Install Visual Studio Code.*** Download it from the official website (https://code.visualstudio.com/) and install it.
2. ***Install MinGW g++.*** Install a MinGW package that includes the g++ compiler, so that g++.exe is available on the computer.
3. ***Add the MinGW bin folder to the Windows PATH.*** Open the Windows search box, type "Edit the system environment variables", open it, click Environment Variables, select Path, click Edit, add the full path of the MinGW bin folder (the folder that contains g++.exe) and click OK. Restart Visual Studio Code afterwards. To check the setup, type g++ --version in a terminal; the compiler version should be displayed.
4. ***Opentheprojectfolder in VS Code.*** Choose File, then Open Folder, and select the folder that contains the game.
5. ***Save the source file as main.cpp.*** Make sure the file name is exactly main.cpp (not main.cpp.txt) and save it with Ctrl + S.
6. ***Open the integrated terminal.*** Choose Terminal, then New Terminal.
7. ***Go to the folder that contains main.cpp.*** Use the cd command. The path below is only an example; change it to the real location of your project folder.

```
cd ".\.vscode\MarioGame"
```

Compile the game with the following command:

```
g++ main.cpp -o MarioAdventure.exe -mwindows -lgdi32 -luser32
```

Run the game with the following command:

```
.\MarioAdventure.exe
```

Meaning of the compile command

Part of the command	Meaning
`g++`	Starts the MinGW C++ compiler, which compiles the code and links it into a program.
`main.cpp`	The source file to be compiled.
`-o MarioAdventure.exe`	Gives the output executable file the name MarioAdventure.exe.
`-mwindows`	Builds a Windows graphical (GUI) program that starts from WinMain, so no console window is opened with the game.

Part of the command	Meaning
`-lgdi32`	Links the Windows GDI library, which provides drawing functions such as `Ellipse`, `CreateSolidBrush`, `CreateFontA`, `TextOutA` and `BitBlt`.
`-luser32`	Links the Windows user-interface library, which provides window, message, timer and keyboard functions such as CreateWindowExA, `SetTimer` and `GetAsyncKeyState`.

The executable file MarioAdventure.exe is created in the same folder from which the compile command is run, which is the folder that contains main.cpp.

If the compiler cannot find main.cpp: check that the terminal is in the correct folder (type dir to list the files and confirm that main.cpp is shown); correct the cd path if necessary and put the path in quotation marks if it contains spaces; make sure the file name is spelled exactly main.cpp with the extension .cpp; and save the file before compiling again.

11. Output Screenshots

The figures below show the Mario Adventure game while it was running. Figures 1 and 3 to 10, 12 and 14 are screenshots captured from the running game. Figures 2, 11 and 13 are placeholders reserved for screenshots that are not yet inserted.

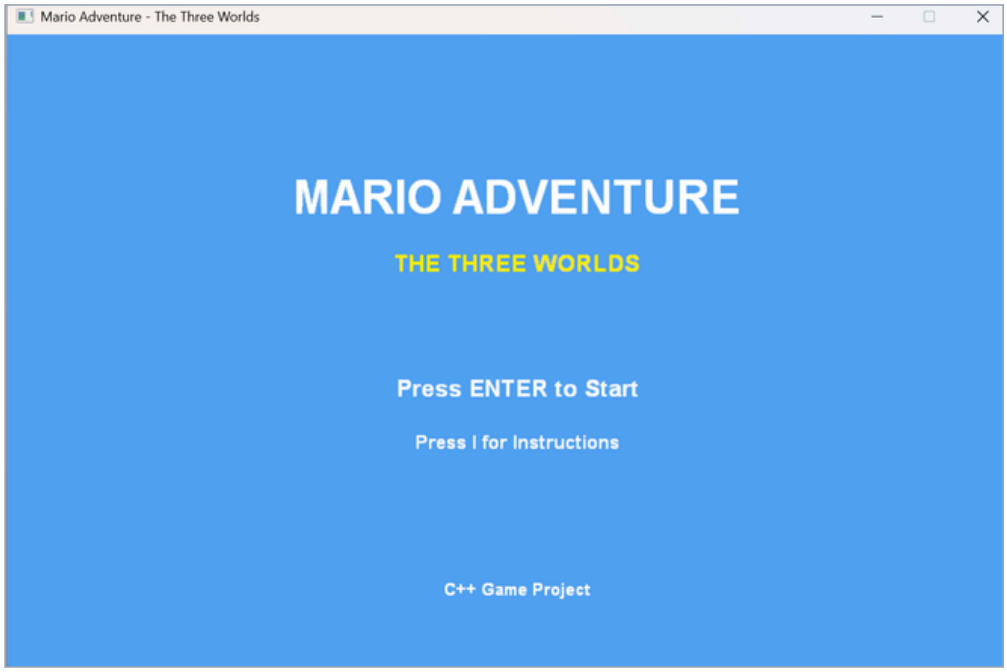


Figure 1: Main Menu Screen. The game title, the message "Press ENTER to Start" and the message "Press I for Instructions" are displayed.

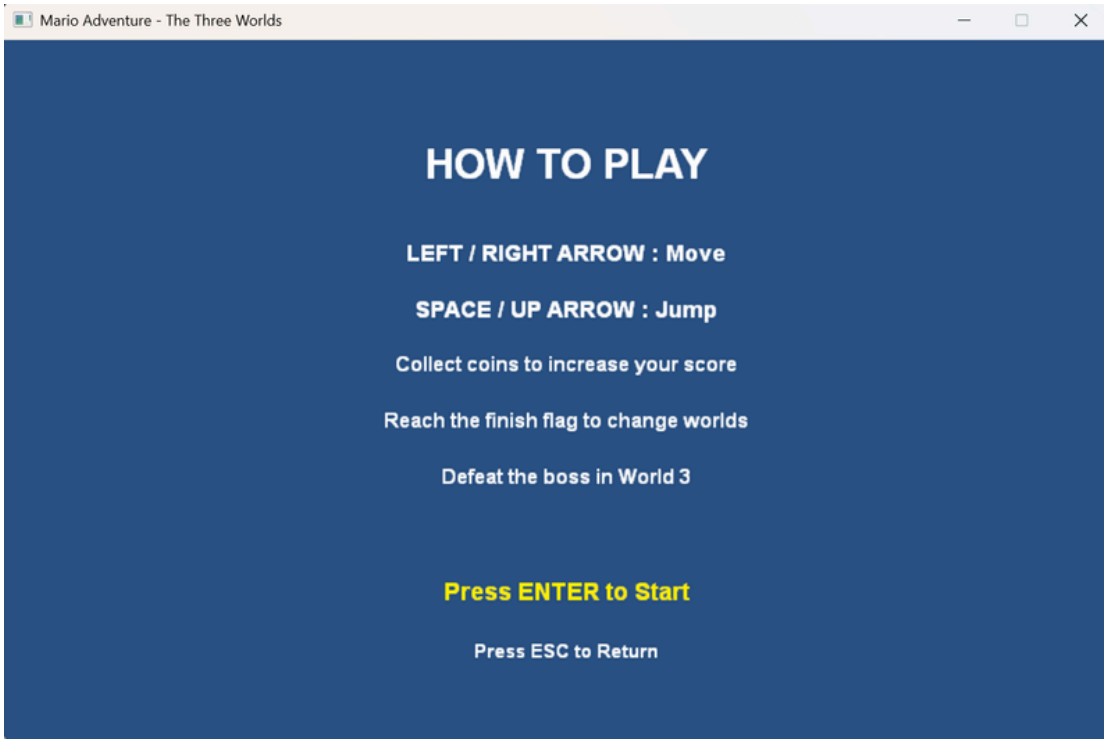


Figure 2: Instructions Screen (HOW TO PLAY), which is opened by pressing I on the main menu.

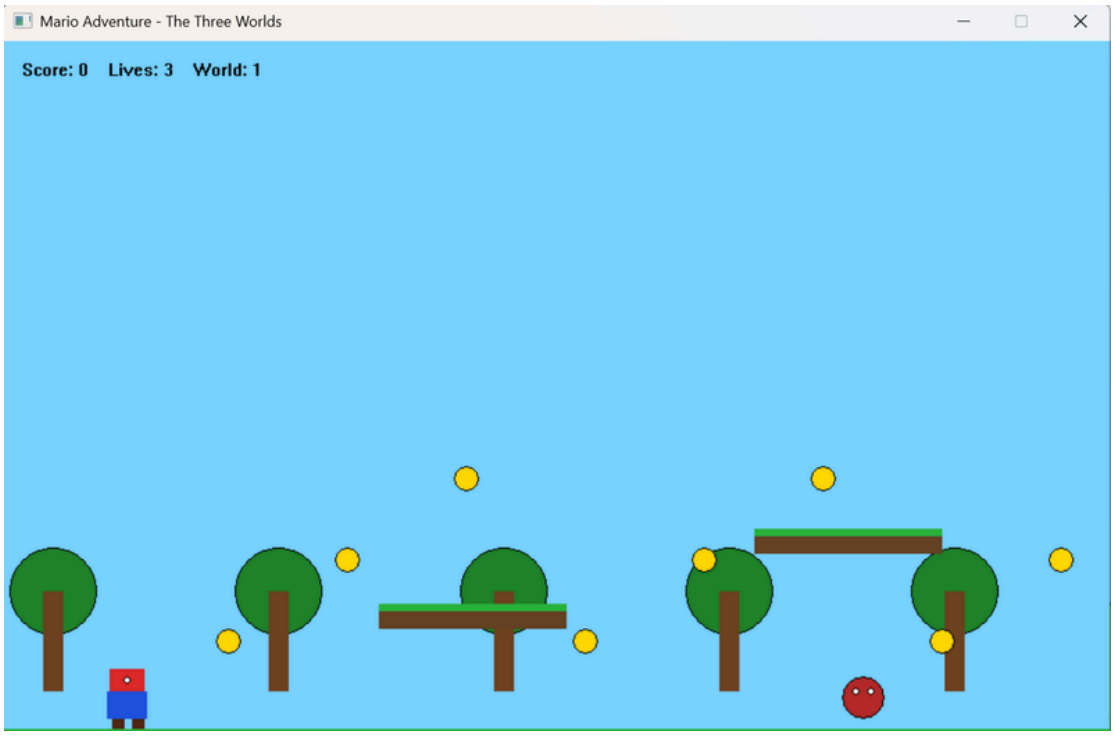


Figure 3: Forest World gameplay at the start of the level: the player, the trees in the background, coins, platforms and a moving enemy, with Score, Lives and World shown at the top-left corner.



Figure 4: Forest World after the checkpoint is reached: CHECKPOINT SAVED is displayed at the top-right and the finish flag is visible near the right side.

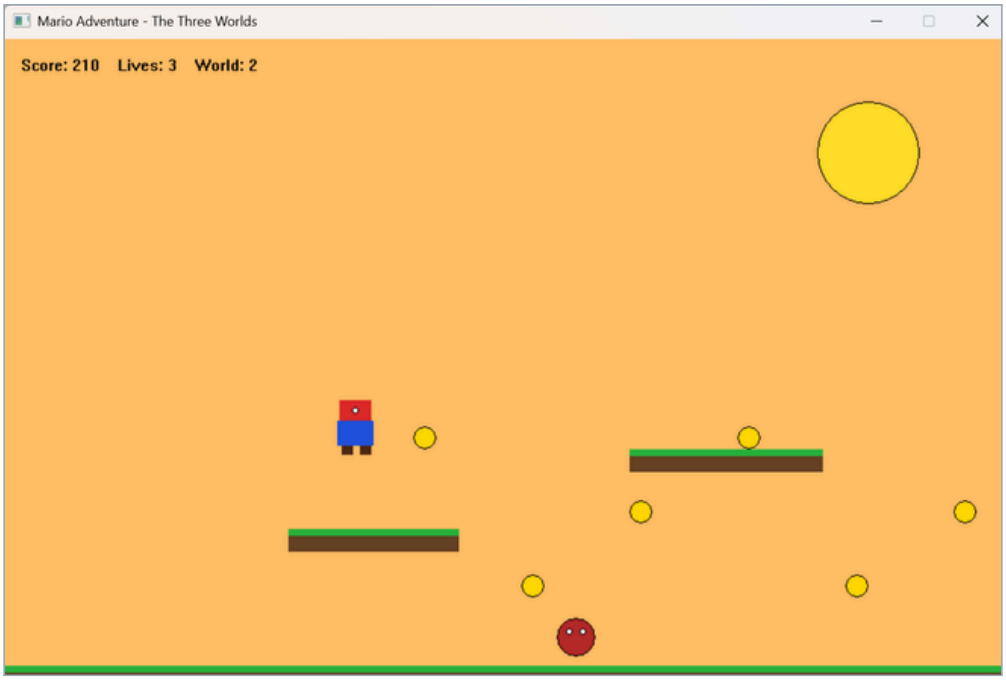


Figure 5: Desert World gameplay: orange sky, sun and a moving enemy, with World 2 shown in the score line.

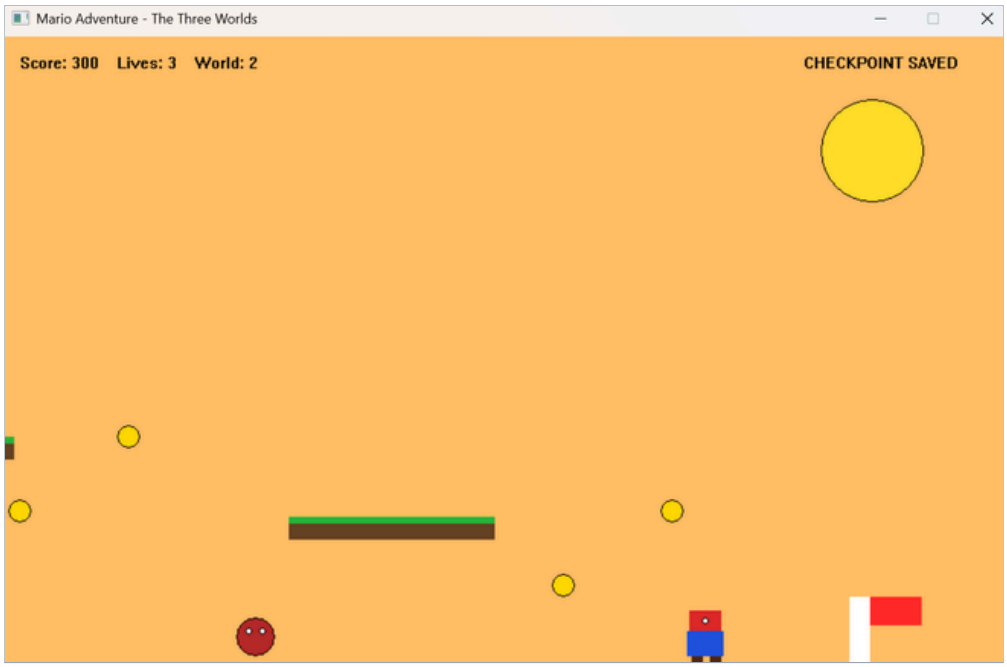


Figure 6: Desert World with the checkpoint saved and the finish flag close to the player.

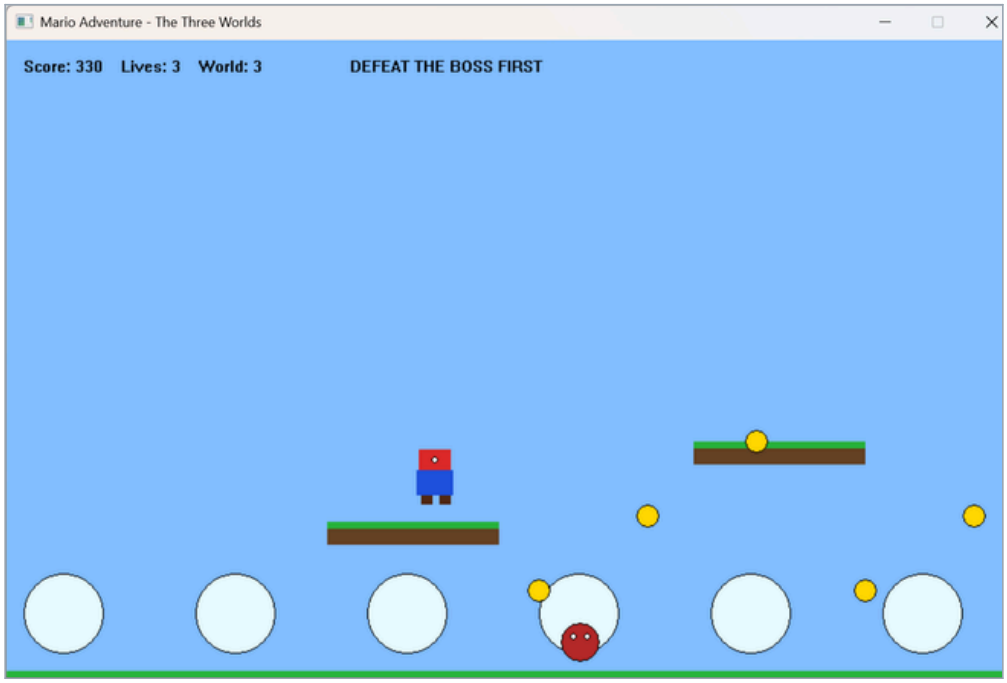


Figure 7: Ice Kingdom gameplay in World 3: pale-blue sky with snow-mound circles, and the message DEFEAT THE BOSS FIRST at the top.

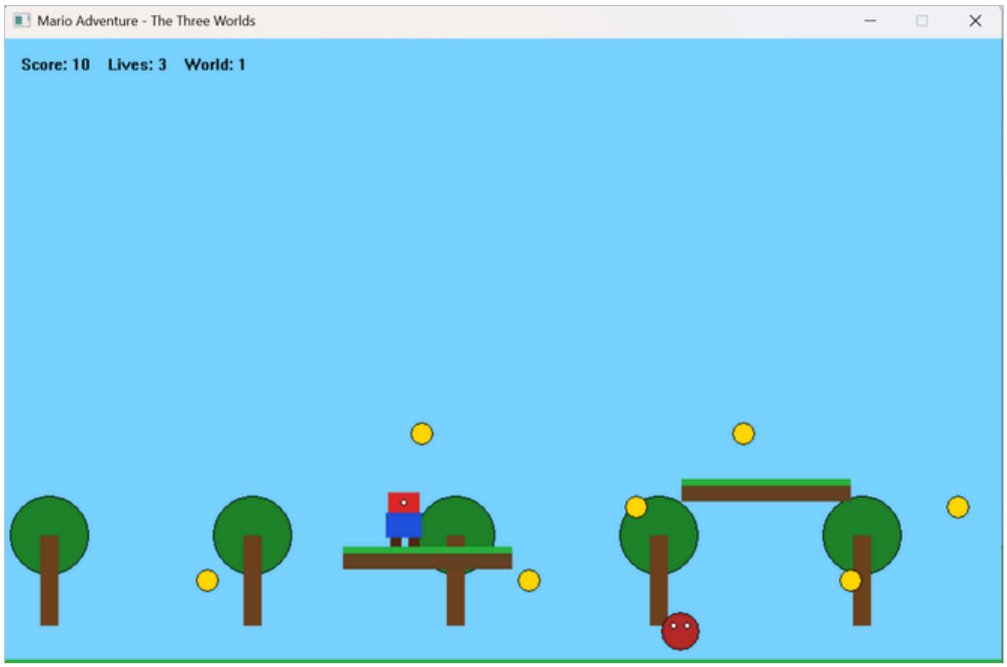


Figure 8: Coin Collection and Score Display: the score shows 10 after the first coin has been collected in Forest World.

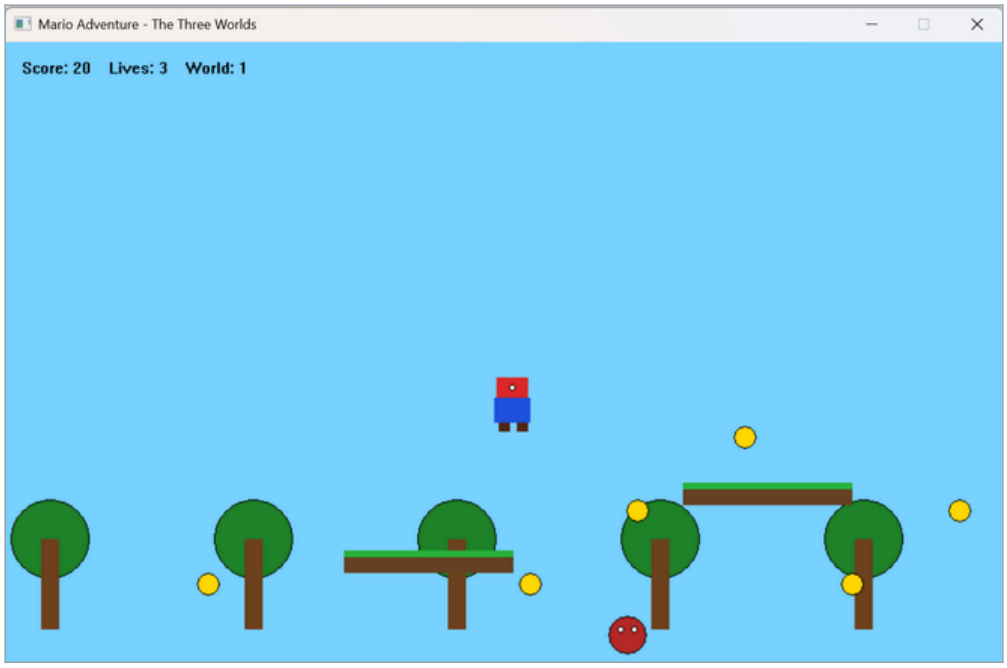


Figure 9: Score display while the player jumps in Forest World; the score has increased to 20 as more coins are collected.



Figure 10: Score display later in Forest World (Score 60), showing the screen scrolling with the player and collected coins removed from the screen.



Figure 11: Power-ups in Gameplay: the name of the active power-up (SPEED POWER, SHIELD POWER or SUPER JUMP) is displayed under the score line.

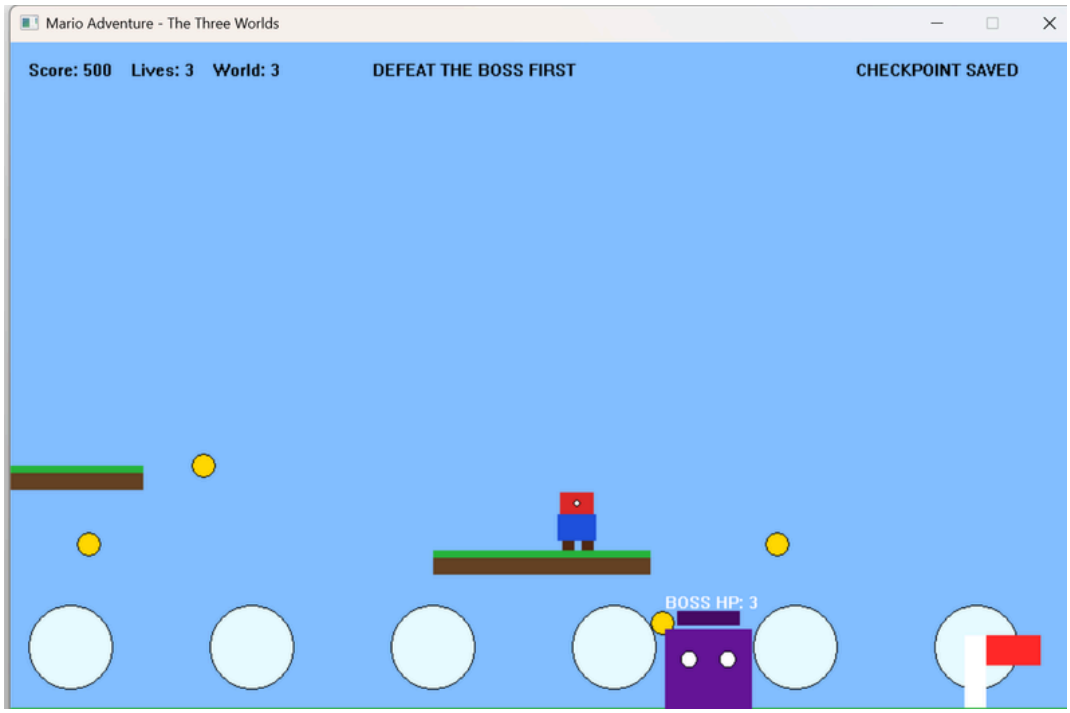


Figure 12: Final Boss Battle in World 3: the boss with BOSS HP: 3 displayed above it, the message DEFEAT THE BOSS FIRST, CHECKPOINT SAVED and the finish flag.

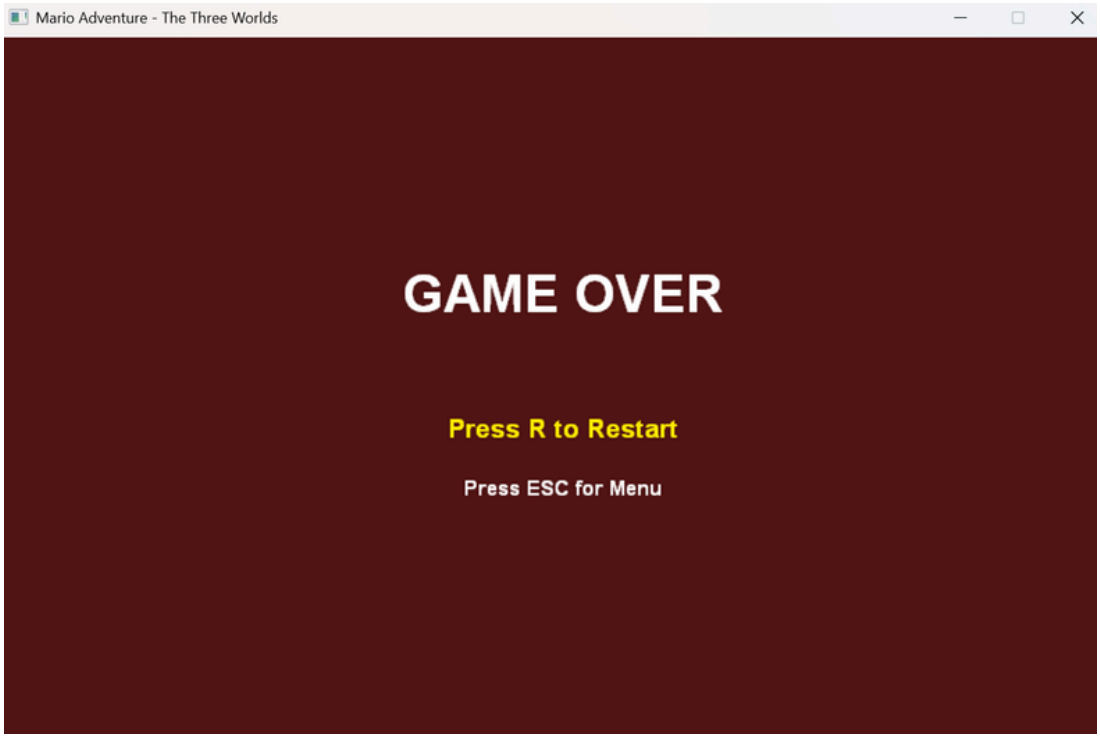


Figure 13: Game Over Screen, which appears when the player loses all three lives.

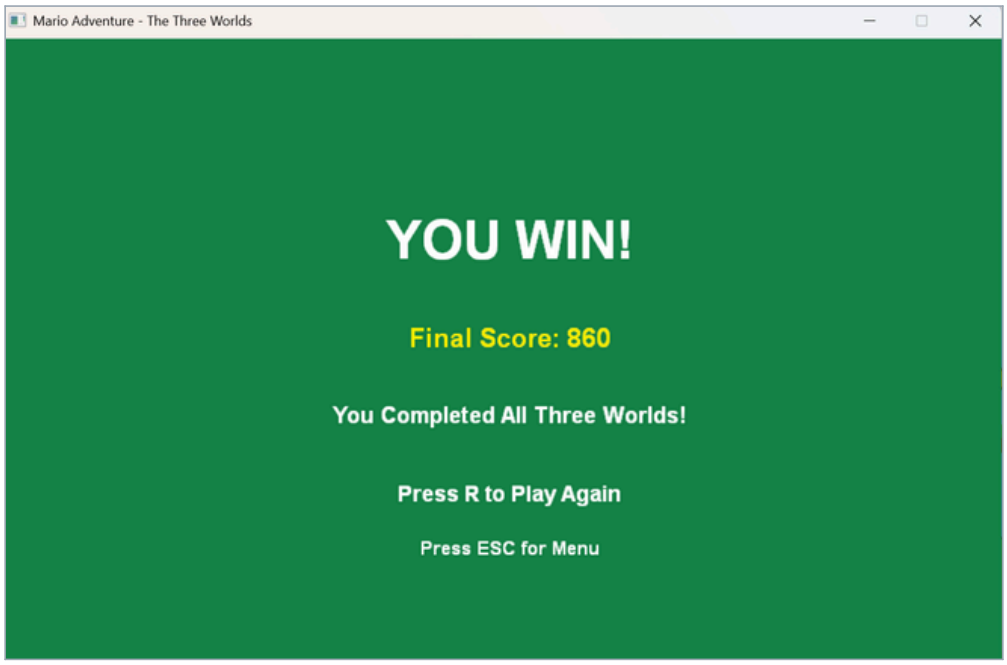


Figure 14: Victory Screen: YOU WIN! with the final score (860 in this run) and the options to play again or return to the menu.

12. Conclusion

Developing ***Mario Adventure – The Three Worlds*** gave practical experience of turning C++ knowledge into a complete working program. The project required the use of variables and constants, conditions, loops, functions, structures, an enumeration and the vector container, and showed how these basic tools can be combined to build a game with several screens and rules.

The author learned how a game loop works, how the position and speed of a character can be changed in every update, and how gravity and jumping can be created with simple arithmetic. Rectangle-based collision detection was used for platforms, coins, enemies and the boss, and it showed how the same idea can solve many different problems. Handling keyboard input with GetAsyncKeyState and window messages was another important part of the work, as was managing game states such as the menu, playing, level complete, game over and win screens.

The project also introduced graphics programming with the Windows API and GDI. Drawing the whole game from rectangles, circles and text, and using an off-screen buffer for smooth display, showed how a window application works in Windows. Testing the game while it was running helped in finding and correcting mistakes in the game logic.

The program is a simple game and is written in a structured style with structures and functions. It does not use classes, inheritance or polymorphism, so it should not be seen as a full object-oriented design. The following ideas are possible future improvements and are ***not*** part of the present version: converting the structures into classes with member functions, adding collectible power-up items in the levels, adding sound effects, adding more worlds and enemy types, and saving the highest score.

13. Bibliography

[1] cppreference.com. ***C++ Reference.***
https://en.cppreference.com/

[2] Standard C++ Foundation. ***Standard C++ (isocpp.org).***
https://isocpp.org/

[3] Microsoft Learn. ***Windows API documentation (Win32).***
https://learn.microsoft.com/en-us/windows/win32/

[4] Microsoft Learn. ***Windows GDI.***
https://learn.microsoft.com/en-us/windows/win32/gdi/windows-gdi

[5] Microsoft Learn. ***GetAsyncKeyState function.***
https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-getasynckeystate

[6] MinGW-w64 Project. ***MinGW-w64 official website.***
https://www.mingw-w64.org/

[7] GNU Project. ***GCC, the GNU Compiler Collection – Online Documentation.***
https://gcc.gnu.org/onlinedocs/

[8] Microsoft. ***Visual Studio Code Documentation.***
https://code.visualstudio.com/docs

[9] Microsoft. ***Using GCC with MinGW – Visual Studio Code.***
https://code.visualstudio.com/docs/cpp/config-mingw

Mario Adventure - The Three Worlds

UML / Class Diagram.

